

1. What is the need for OOPs? or What are some advantages of using OOPs? or Why is OOPs so popular?

- OOPs helps users to understand the software easily, although they don't know the actual implementation.
- With OOPs, the readability, understandability, and maintainability of the code increase multifold.
- Even very big software can be easily written and managed easily using OOPs.
- OOPs also helps to hide the unnecessary details with the help of Data Abstraction.
- Polymorphism offers a lot of flexibility in OOPs.
- As a result of these, OOPs is so popular.

In []:

2. What is a class?

- A class can be understood as a template or a blueprint, which contains some values, known as member data or member, and some set of rules, known as behaviors or functions.
- So when an object is created, it automatically takes the data and functions that are defined in the class.
- Therefore the class is basically a template or blueprint for objects.
- Also one can create as many objects as they want based on a class. For example, first, a car's template is created. Then multiple units of car are created based on that template.

```
In [9]: class car:           #class
        def specifications(self,name,speed):      #object
            self.name=name
            self.speed=speed
            print("The car name is",self.name,",color is","and speed is",self.speed)
obj1=car()
obj1.specifications("Ferrari","340KMPH")
```

The car name is Ferrari ,color is and speed is 340KMPH

In []:

3. What is class constructor

- The constructor is a method that is automatically gets called every time when an object is created.
- This method is defined in the class and can be used to initialize basic variables.
- Adding attributes to the object manually after creating it is not a good practice,instead python offers a much more elegant and compact way of defining attributes right while instantiating the object. for that we use **init** method
- The **init()** function is called automatically every time the class is being used to create a new object.

```
In [3]: class car:
        def __init__(self,name,color,speed):
            self.name=name
            self.color=color
            self.speed=speed
        def specification(self):
            print("The car name is",self.name,",color is",self.color)
        def speed1(self):
            print("and speed is",self.speed)

obj1=car("Ferrari","black","340KMPH")
obj1.specification()
obj1.speed1()
```

The car name is Ferrari ,color is black
and speed is 340KMPH

In []:

4. What is an object?

- An object refers to the instance of the class, which contains the instance of the members and behaviors defined in the class template.
- In the real world, an object is an actual entity to which a user interacts, whereas class is just the blueprint for that object.
- So the objects consume space and have some characteristic behavior. For example, a specific car

```
In [4]: class Person:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        def myfunc(self):
            print("Hello my name is " + self.name)

p1 = Person("John", 36)      #object name p1 of class Person is created
p1.myfunc()                 #Object called
```

Hello my name is John

In []:

5.what is self parameter?

The self parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

It does not have to be named self , you can call it whatever you like, but it has to be the first parameter of any function in the class:

In []:

6. What is encapsulation

- One can visualize Encapsulation as the method of putting everything that is required to do the job, inside a capsule and presenting that capsule to the user. --What it means is that by Encapsulation, all the necessary data and methods are bind together and all the unnecessary details are hidden to the normal user.
- So Encapsulation is the process of binding data members and methods of a program together to do a specific job, without revealing unnecessary details.
- The way that data and methods are structured does not matter to the end user.

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

Public Member (accessible within or outside of a class)

```
        self._project = project
```

Protected Member (accessible within the class and it's sub-classes)

```
        self.__salary = salary
```

Private Member (accessible only within a class)

Data Hiding using Encapsulation

```
In [5]: #public member
class bike:
    name="royal enfield" #public member

    def ride(self): #public member
        print("live free ride hard")

obj=bike()
obj.ride()
obj.name
```

```
live free ride hard
```

```
Out[5]: 'royal enfield'
```

```
In [6]: #protected member
class base:
    def __init__(self):
        self._a=2    #protected member
class derived(base):
    def protected(self):
        print("calling protected member of base class",self._a)
obj=derived()
obj.protected()
obj._a
```

calling protected member of base class 2

Out[6]: 2

```
In [7]: #private member
class base:
    a=5
    def __init__(self):
        self.__a=2    #private member
class derived(base):
    def private(self):
        print("calling private member of base class",self.__a)
obj=derived()
obj.private()
```

AttributeError Traceback (most recent call last)

Input In [7], in <cell line: 10>()
 8 print("calling private member of base class",self.__a)
 9 obj=derived()
 ----> 10 obj.private()

Input In [7], in derived.private(self)
 7 def private(self):
 ----> 8 print("calling private member of base class",self.__a)

AttributeError: 'derived' object has no attribute '_derived__a'

In []:

7. What is Polymorphism?

- Polymorphism is composed of two words - "poly" which means "many", and "morph" which means "forms". Therefore Polymorphism refers to something that has many forms.
- In OOPs, Polymorphism refers to the process by which some code, data, method, or object behaves differently under different circumstances or contexts.
- polymorphism can be implement using three ways
- 1) operator overloading
- 2) method overloading
- 3) method overriding

In []:

8. What is the difference between overloading and overriding?

- Method Overloading is an example of Compile time polymorphism. In this, more than one method of the same class shares the same method name having different parameters. Method overloading is used to add more to the behavior of methods and there is no need of more than one class for method overloading.
- Method overriding is an example of run time polymorphism. In this, the specific implementation of the method that is already provided by the parent class is provided by the child class. It is used to change the behavior of existing methods and there is a need for at least two classes for method overriding. In method overriding, inheritance always required as it is done between parent class(superclass) and child class(child class) methods.
- The method Overriding occurs between superclass and subclass. Overloading occurs between the methods in the same class.

```
In [ ]: #method overLoading
class OverloadingExample:
    def add(self,a,b):
        print(a+b)
    def add(self,a,b,c):
        print(a+b+c)
a=OverloadingExample()
a.add(5,10)
a.add(5,10,20)
```

- Note: Python does not support method overloading, this is because python always picks up the latest defined method. We can still overload methods in python but it would be of no use. However, you can implement method overloading in the above way in Java, C++, etc.
- An alternative to performing method overloading in Python would be in this way:

```
In [ ]: class OverloadingExample:
    def add(self,a,b):
        print(a+b)
    def add(self,a,b,c=4):
        print(a+b+c)
a=OverloadingExample()
a.add(5,10)
a.add(5,10,20)
#Here, we don't mention the function definition multiple times.
#Instead, we manipulate the arguments using the same method repeatedly.
```

```
In [ ]: #method overriding- same method name in the parent class behaves differently in i
class bird:
    def intro(self):
        print("there are different types of birds")
    def flight(self):
        print("most of the birds can fly bt some cannot")
class parrot(bird):
    def flight(self):
        print("parrots can fly")
class penguin(bird):
    def flight(self):
        print("penguin do not fly")
obj=bird()
obj.intro()
obj.flight()
obj1=parrot()
obj1.flight()
obj2=penguin()
obj2.flight()
```

In []:

9. what is operator overloading?

- Operator Overloading means giving extended meaning beyond their predefined operational meaning. For example operator + is used to add two integers as well as join two strings and merge two lists. It is achievable because '+' operator is overloaded by int class and str class. You might have noticed that the same built-in operator or function shows different behavior for objects of different classes, this is called Operator Overloading.

```
In [ ]: # Python program to show use of
# + operator for different purposes.

print(1 + 2)

# concatenate two strings
print("data"+ "science")

# Product two numbers
print(3 * 4)

# Repeat the String
print("python"*3)
```

In []:

10. what is data abstraction?

- If you are a user, and you have a problem statement, you don't want to know how the components of the software work, or how it's made. You only want to know how the software solves your problem.

- Abstraction is the method of hiding unnecessary details from the necessary ones. It is one of the main features of OOPs.
- For example, consider a car. You only need to know how to run a car, and not how the wires are connected inside it. This is obtained using Abstraction.

Abstraction in python:

- 1)Abstract class: A class which contains one or more abstract methods is called an abstract class.
- 2)Abstract method An abstract method is a method that has a declaration but does not have an implementation.
- 3)concrete class: A class which doesnt have any abstract method.

```
In [ ]: from abc import ABC, abstractclassmethod
class my_class: # concrete class
    def study(self):
        print("Studing python")

obj1=my_class()
obj1.study()
```

```
In [ ]: class my_class1(ABC):
    def study(self):
        print("Studing python")

    @abstractclassmethod    # decorator:to make abstract method
    def mock(self):        #a method with only declaration but no implementation
        pass

obj=my_class1()
```

Note:

- we cant create or instantiate object of abstract class.
- object can only be created for concrete class.
- so to achive abstraction we need :
- 1)Inherhit the ABC class for your base class to be abstract class
- 2)Create a method with no implementation and only declaration
- 3)use a @abstractclassmethod decorator to make your method as abstract method

```
In [ ]: class my_class1(ABC): #abstract class

    def study(self):
        pass

    @abstractclassmethod      #abstract method
    def mock(self):
        pass
class student(my_class1):    #concrete class
    def miss_mock(self):
        print("enjoying the weekend to fullest")

    def mock(self):
        print("doing mock to become DS")

    def study(self):
        pass

s1=student()
s1.mock()
s1.miss_mock()
s1.study()
```

11. what is super method in abstraction?

- using super() method we can invoke the methods in abstract classes

```
In [ ]: class Animal(ABC):    #abstract class

    def sleep(self): # concrete method
        print("i am going to sleep")

    @abstractclassmethod
    def sound(self): #abstract method
        pass

class snake(Animal):
    def sound(self):
        print("hissssing sound")
        super().

s=snake()
s.sound()
```

In []:

In []:

12. Difference between encapsulation and abstraction.

DATA ABSTRACTION VERSUS ENCAPSULATION

DATA ABSTRACTION	ENCAPSULATION
OOP concept that hides the implementation details and shows only the functionality to the user	OOP concept that binds or wraps the data and methods together into a single unit
Hides the implementation details to reduce the code complexity	Hides data for the purpose of data protection
OOP languages use abstract classes and interfaces to achieve Data Abstraction	OOP languages can achieve Encapsulation by making the data members private and accessing them through public methods
	Visit www.PEDIAA.com

13. Why abstraction is important?

- Data Abstraction firstly saves a lot of our time as we do not have to repeat the code that may be the same for all the classes.
- Moreover, if there are any additional features, they can be easily added, thus improving flexibility.
- Not to mention, working in large teams becomes easier as one won't have to remember every function and the basic structure can be inherited without any confusions.
- While we are designing large functional units we use an abstract class.
- When we want to provide a common interface for different implementations of a component, we use an abstract class.

In []:

14. What are the types of class variables? there are three types of variables:

- 1)instance variable(object level variable)
- 2)static variable(class level variable)
- 3)local variable(method level variable)

1) instance variable : -when value of variable is changing from object to object then such variables are called instance variable -declared within the methods or object

2) Static variable: -when value of variable is not changing from object to object then such variables are called static variable -declared inside a class but outside any method -we can call them using class name

3) Local variables: -a variable that is defined in a block is available only in that block.it is not accessible outside the block

```
In [ ]: class cricket:
    Team="India" #static variable

    def __init__(self,name,jersey_no):
        self.name=name
        self.jersey_no=jersey_no #instance variable

    def IPL(self):
        x="RCB" #Local variable
        print("the winner of ipl is ",x)
        print("instance variable is :",self.name) #self.name instance variable

obj=cricket("MS dhoni",7)
print(obj.name)
obj.IPL()
cricket.Team #static variable access by using class name
```

In []:

15 what is Inheritance?

- Python Inheritance Inheritance allows us to define a class that inherits all the methods and properties from another class.
- Parent class is the class being inherited from, also called base class. also called Base class
- Child class is the class that inherits from another class, also called derived class. also called Derived Class

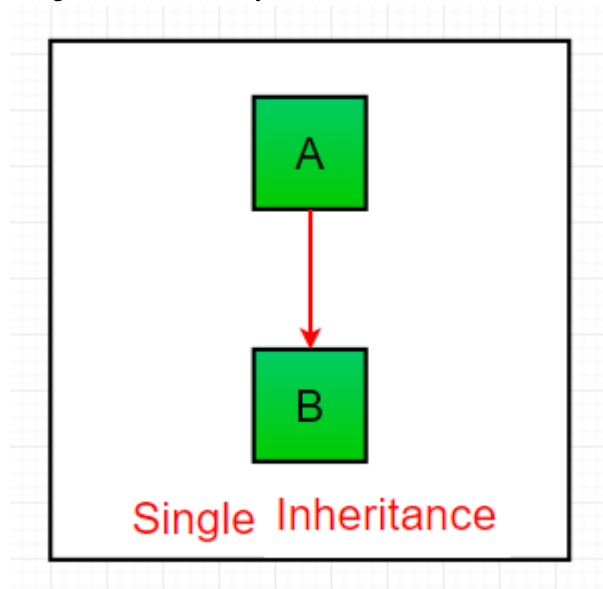
In []:

16. Types of inheritance

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance

- Hybrid inheritance

1)single inheritance- Single inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code.



```
In [ ]: #single inheritance

class A: # parent class
    def feature1(self):
        print("feature 1 is working")

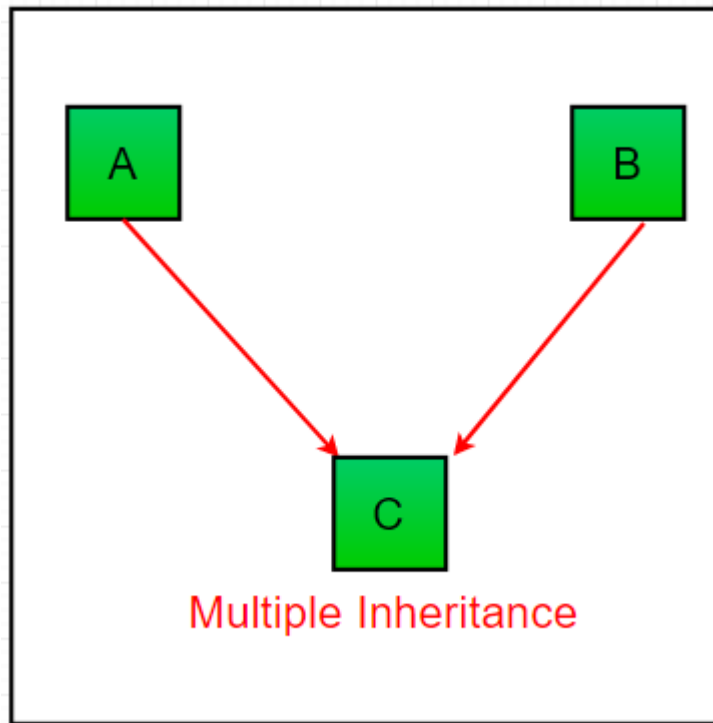
    def feature2(self):
        print("feature 1 is working")

class B(A):#child class
    def feature3(self):
        print("feature 3 is working")

    def feature4(self):
        print("feature 4 is working")

b=B()
b.feature1()
```

2)Multiple inheritance When a class can be derived from more than one base class this type of inheritance is called multiple inheritances. In multiple inheritances, all the features of the base classes are inherited into the derived class.



```
In [ ]: #multiple inheritance
class A: # parent class1
    def feature1(self):
        print("feature 1 is working")

    def feature2(self):
        print("feature 2 is working")

class B: #Parent class 2

    def feature3(self):
        print("feature 3 is working")

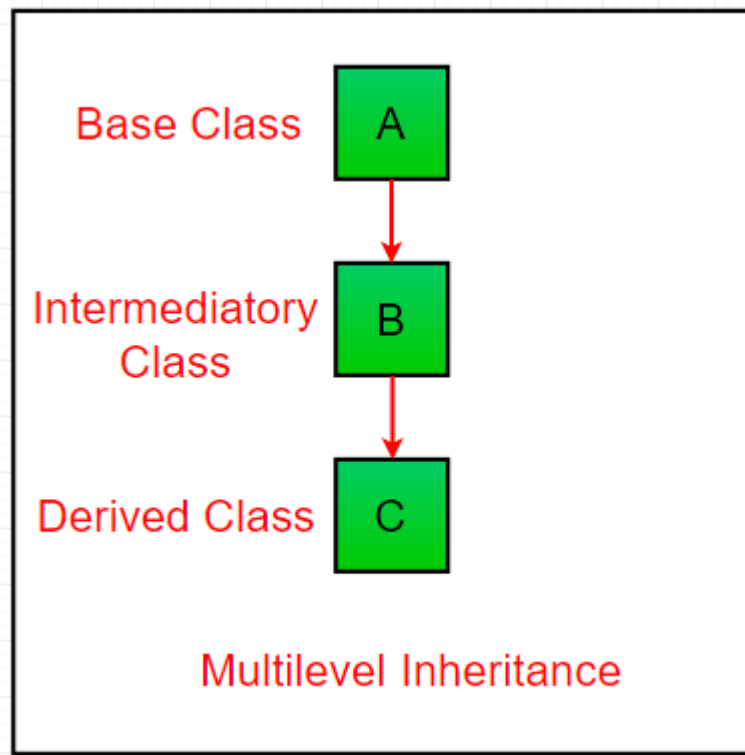
    def feature4(self):
        print("feature 4 is working")

class C(A,B): #child class
    def feature5(self):
        print("feature 5 is working")

    def feature6(self):
        print("feature 6 is working")

c=C()
c.feature2()
c.feature3()
c.feature6()
```

3)Multilevel Inheritance : In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to a relationship representing a child and a grandfather.



```
In [ ]: #multilevel inheritance
class A: # parent class
    def feature1(self):
        print("feature 1 is working")

    def feature2(self):
        print("feature 2 is working")

class B(A):#child class
    def feature3(self):
        print("feature 3 is working")

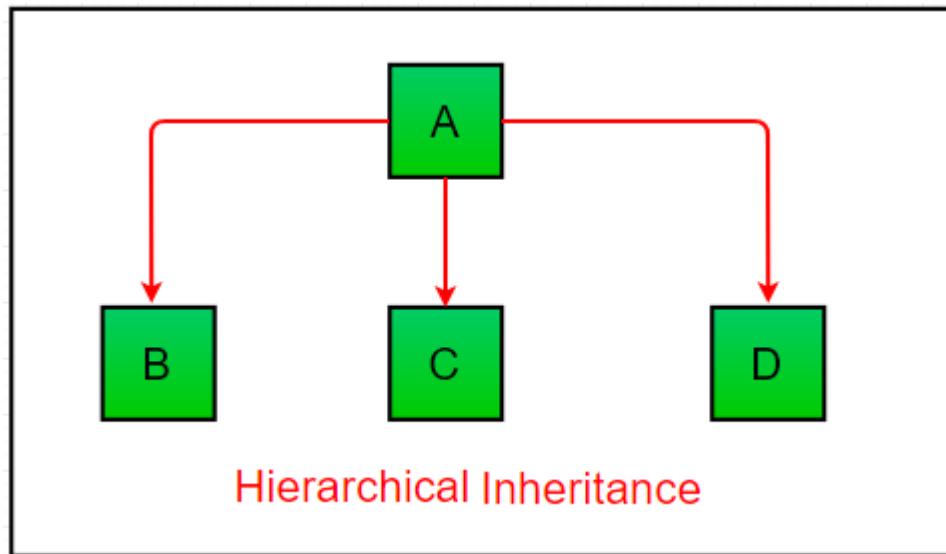
    def feature4(self):
        print("feature 4 is working")

class C(B):#child class
    def feature5(self):
        print("feature 5 is working")

    def feature6(self):
        print("feature 6 is working")

c=C()
c.feature1()
c.feature3()
c.feature6()
```

4) Hierarchical Inheritance When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes.



```
In [ ]: #Hierarchical inheritance
class A: # main parent
    def feature1(self):
        print("feature 1 is working")

    def feature2(self):
        print("feature 2 is working")

class B(A):#child 1

    def feature3(self):
        print("feature 3 is working")

    def feature4(self):
        print("feature 4 is working")

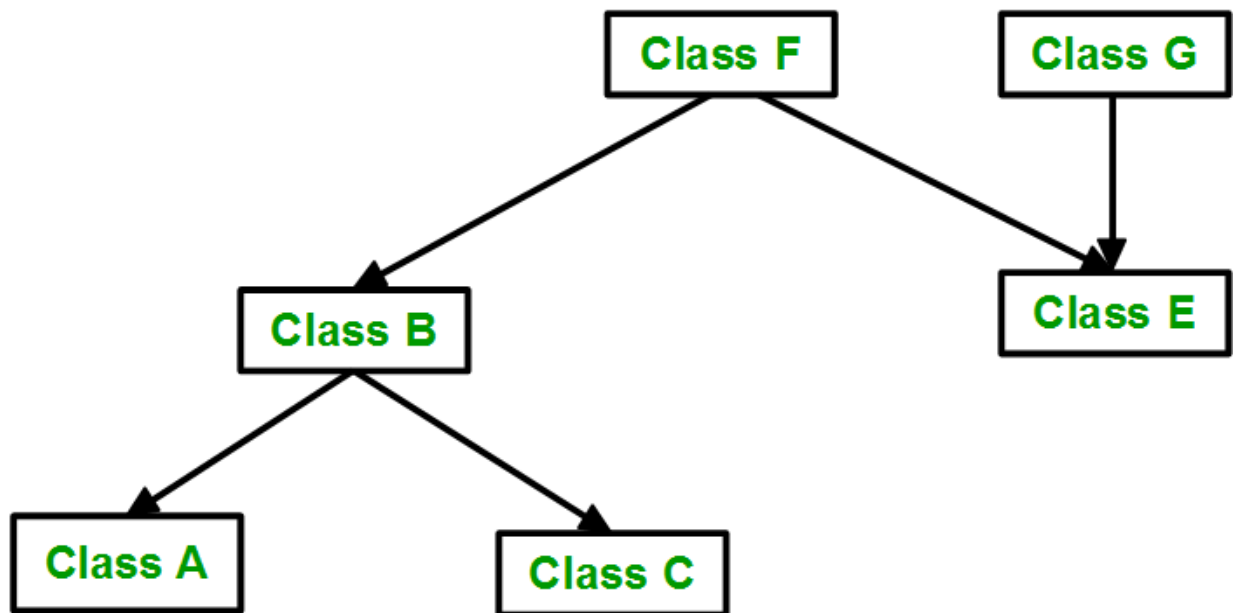
b=B()
b.feature2()

class C(A):#child 2
    def feature5(self):
        print("feature 5 is working")

    def feature6(self):
        print("feature 6 is working")

c=C()
c.feature5()
```

5)Hybrid inheritance Inheritance consisting of multiple types of inheritance is called hybrid inheritance.



```
In [ ]: #Hybrid inheritance

class A: # main parent
    def feature1(self):
        print("feature 1 is working")

    def feature2(self):
        print("feature 2 is working")

class B(A): #child 1

    def feature3(self):
        print("feature 3 is working")

    def feature4(self):
        print("feature 4 is working")

b=B()
b.feature2()

class C(A): #child 2
    def feature5(self):
        print("feature 5 is working")

    def feature6(self):
        print("feature 6 is working")

c=C()
c.feature5()

class D(B,C):#child 3
    def feature7(self):
        print("feature 7 is working")

    def feature8(self):
        print("feature 8 is working")

d=D()
d.feature2()    #class D inherits all the properties from clas A,B,C and D itself
```

In []:

17. Difference between method and constructor?

- Constructor is used to initialize an object whereas method is used to exhibits functionality of an object.
- Constructors are invoked implicitly whereas methods are invoked explicitly.
- Constructor does not return any value where the method may/may not return a value.
- In case constructor is not present, a default constructor is provided by java compiler. In the case of a method, no default method is provided.
- Constructor should be of the same name as that of class. Method name should not be of the same name as that of class.

In []:

In []: